

Converge

THE CANONICAL BLUEPRINT · V3 – WITH LOOP ENGINEERING

The Eight Passes — Compile Intent, *Close the Loop.*

The canonical blueprint for compiling a fuzzy brief into an autonomous, eval-gated system — **eight passes, one fork, and a closed loop** that drives a fleet to green. Build the factory from day one; walk out one gate at a time.

8

PASSES + 1
FORK

11

CLAUDE CODE
SKILLS

2

MOATS, KEPT
APART

1

DEFINITION OF
DONE

DOCUMENT

Canonical process blueprint · v3

CONVERGED MEANS

an eval passed — not that you feel done

The invariant — every pass lowers altitude, binds an engine, ends at a gate. Pass 8 is the exception: it does not lower, it closes. Loop Engineering — designing the loops that prompt agents rather than prompting agents — is placed here as the eighth pass that closes the spine.

Introduction

Why Converge exists

The old loop ran in your head. A requirement arrived fuzzy, and you held the definition of "done" as private judgment — refined by taste, defended in review, never written down. That loop does not scale to AI engines, because an engine cannot execute a standard it cannot read. The bottleneck was never typing speed; it was that intent lived in a human and had to be re-supervised at every step. The AI-native engineer inverts this: take a fuzzy requirement and lower it through a repeatable descent — each step more precise than the last — until "done" is a machine-checkable condition an engine can satisfy unsupervised.

The main idea — compile intent, don't write the system

Stop writing the system. Compile it. Converge treats building as a **compilation pipeline** that lowers altitude in fixed stages: intent → structure → plan → tasks → code → loop. Each stage is a transformation with a typed output and a gate that must pass before the next stage runs — exactly how a compiler lowers source through intermediate representations down to machine code. You author the high-level intent and the checks; the pipeline emits the system. Your role shifts from author to compiler — you specify the descent and verify each lowering, rather than hand-writing the final artifact.

The thesis — build the factory from day one

You do not transform a workflow into a factory later. That retrofit — bolting evals, harnesses, and loops onto human-centric code never built to be gated — is the slow, expensive path, and it is the path almost everyone takes. Converge refuses it. The framework is factory-shaped from Pass 1: every pass already ends in a gate, every task is born with a runnable eval, and the harness is a control plane from the first file. So when you decide to step out, there is nothing to retrofit and nothing to rewrite; the gates already hold without you, and you walk out one gate at a time.

EVERY PASS

Lowers altitude

intent → structure → plan →
tasks → code → loop. Each step
closer to something a machine
runs.

BINDS

An engine

the right tool for the altitude —
Cowork to think, Claude Code
to build, an adversary to attack,
a fleet to crank.

ENDS AT

A gate

converged means an eval
passed — not that you feel
done. The gate is the unit of
trust.

THE INVARIANT

Every pass lowers altitude, binds an engine, ends at a gate. Pass 8 is the exception — it does not lower, it **closes**. You are converged when the eval passes — not when you feel done.

Contents

—	Introduction	What Converge is, and why
0	The Whole Method, On One Line	the flat table + the invariant
—	The Spine, Drawn	eight passes, one fork
1	Intent — brd-docs-to-tech-req	altitude: intent
2	Structure — tech-req-to-adrs	altitude: system
3	Decompose — reqs-to-swimlane-plans	altitude: plan
4	Consensus — sketch-plans-adversarial-review	the fork is decided
—	The Fork — one big gate, or many small ones?	+ when to use each
5A	Specify — plans-to-coherent-spec	Fork A · plan-driven
5B	Tasking — task-spec	Fork B · task-driven
①	Register — task-specs-to-issues	the tracker as state
6	Harness — stack-to-harness	the quality moat
8	The Loop — task-loop; Manager = future CI/CD	altitude: runtime
—	Why This Is Defensible	two moats · positioning · honest boundary
—	The Runtime, Proven	what the skills already enforce
—	One Method, Every Engineer	the role lives in the eval
—	The Method, End to End on One Feature	the worked example



The Whole Method, On One Line

Intent → Structure → Decompose → Consensus → [Fork: Specify | Tasking] → Register
→ Harness → Loop

A fuzzy brief becomes a tech-spec, the tech-spec becomes ADRs, the ADRs become swimlane plans, the plans survive an adversarial review — and there the path forks on where the trust boundary sits. Specify wraps the whole system; Tasking wraps each unit. Both branches reconverge through the same Harness and Loop. Each output is the next input; the chain descends from a fuzzy brief to a fleet running green.

#	PASS	SKILL	OUT · GATE
1	Intent	brd-docs-to-tech-req	tech-spec · answers the brief
2	Structure	tech-req-to-adrs	docs/adrs/* · system understood
3	Decompose	reqs-to-swimlane-plans	swimlane plans · plan-altitude only
4	Consensus	sketch-plans-adversarial-review	plans sharpened · fork decided
♦	The Fork	— decision —	where does the trust boundary sit?
5A	Specify	plans-to-coherent-spec	one coherent spec · one e2e eval
5B	Tasking	task-spec	many units · each has an eval
①	Register	task-specs-to-issues	1 spec = 1 issue · the board is state
6	Harness	stack-to-harness	.claude/ + AGENTS.md · control plane
8	The Loop	task-loop --issue N	each task → green eval → PR

THREE GUARDRAILS MAKE THE SEQUENCE CANONICAL

Pass 2 writes ADRs only (no code). **Pass 3 is plan-altitude only** (no tasks, no SQL). **The fork is an output of Pass 4** — never assumed earlier. And Pass 8 does not lower — it closes.

The spine, drawn — eight passes, one fork

THE FULL DESCENT, AS A DECISION TREE

Four passes lower altitude in a straight line. At Consensus the path forks into 5A · Specify and 5B · Tasking — and both branches reconverge through Harness, Execute, and the Loop. The fork is the only branch in the whole descent; everything above it is linear, everything below it merges again.



FIG 1 The 8-pass descent as a decision tree – linear until Consensus, where the single fork splits into 5A (Specify) and 5B (Tasking). Both reconverge at Harness; Pass 8 closes the loop. The double-ruled boxes mark the two fork branches – the only place the spine is not a straight line.

1 The client hands you the problem. You produce the spec.

PASS 1 OF 8 · ALTITUDE: INTENT · *receive & comprehend the brief*

ENGINE · CLAUDE COWORK

SKILL · brd-docs-to-tech-req

OUT · **TECH-SPEC**

Read the BRD like a senior engineer, not a stenographer. Find the real problem underneath the ask, name who actually hurts when it is unsolved, and pin the numbers that define success. Ambiguity left here compounds through every pass downstream — this is the cheapest place in the whole spine to kill it, and the most expensive to leave alone.

UNDERSTAND

Read

the BRD like a senior engineer — the real problem underneath, who hurts, the numbers that define success.

INTERROGATE

Grill

the 2–3 questions whose answers most change how this gets built; ambiguity here compounds downstream.

CRYSTALLIZE

Spec

resolve the open questions into one technical answer — the tech-spec that says what gets built and how success is measured.

GATE

You can restate the client's problem in one breath and show the tech-spec answers it — scope, data, and success metric named, every requirement falsifiable, metrics traced to the BRD's KPIs. The tech-spec is the artifact; clearing the gate is the judgment that it truly answers the brief.

2

Greenfield or brownfield — and write the ADRs.

PASS 2 OF 8 · ALTITUDE: SYSTEM · *ground the spec, record the decisions*

ENGINE · CLAUDE CODE · ON THE REPO

SKILL · tech-req-to-adrs

OUT · docs/adrs/*.md

Before any plan, name the ground. **Brownfield** is a system you did not write: open it end to end, and check the spec against the real schemas, sources, and jobs you find. **Greenfield** is a system that does not yet exist: confirm there is nothing to inherit, then ground the spec in the source systems and constraints it must honour. Either way you leave Pass 2 knowing the terrain — what is actually there, and what the work must respect.

NAME THE GROUND

Field

brownfield (a system you didn't write) vs greenfield (nothing yet) — name it explicitly; the rest follows.

GROUND

Check

open the system end to end; reconcile the spec against real schemas, sources, and jobs.

RECORD

ADRs

write docs/adrs/*.md — grounding decisions (facts + constraints), never how to build. That's Pass 3.

GATE

The system is understood and its grounding decisions are recorded durably, so Pass 3 can read them without re-deriving the terrain. An ADR records what is *true* — schemas, joins, constraints — never how to build. The moment an ADR says "build X", you've drifted into planning.

3

Split the work along its natural seams.

PASS 3 OF 8 · ALTITUDE: PLAN · *decomposition · break it into swimlanes*

ENGINE · CLAUDE CODE · SAME SESSION

SKILL · reqs-to-swimlane-plans

IN · READS THE ADRs

Take the system understood in Pass 2 — same session, ADRs in hand — and split the work along its natural seams. Read the ADRs, then cut where the system already wants to be cut: by feature or by component. Each seam becomes one swimlane; each swimlane gets its own focus-oriented sketch plan. The common case is two lanes — transform versus serve, a medallion lane and a serving lane — but a system carrying several features splits into several lanes. The right number of lanes is the number of genuine seams. No more, no fewer.

DECOMPOSE**Seams**

cut where the system is already jointed — a feature edge or a component edge, not an arbitrary slice.

SWIMLANE**One plan each**

every seam becomes one swimlane carrying exactly one sketch plan — one lane, one plan, one focus.

GROUNDING**Reads ADRs**

each lane inherits the architecture decisions in docs/adrs/ — it builds on Pass 2, not around it.

GATE

One sketch plan per swimlane, each listing features, dependencies, and a sane build order, inheriting the ADR decisions. Plan-altitude only — no atomic tasks, no code. The session carries from Pass 2; the understanding becomes the plans without leaving the session.

REFERENCE SHAPE — THE COMMON TRANSFORM → SERVE CASE

One concrete instance of the seams Pass 3 splits into: a transformation lane (medallion: raw → bronze → silver → gold) feeding a serving lane (MCP / API over gold). Other systems split by feature or component — this is the shape, not the rule.

4

Bring a different engine to refute the plans.

PASS 4 OF 8 · ALTITUDE: PLAN (HARDENED) · *consensus* · *a different model attacks*

ENGINE · ADVERSARY (--adversary codex)

SKILL · sketch-plans-adversarial-review

A single model won't refute itself hard enough. So Pass 4 brings a *different* engine to attack the plans, one at a time, as a skeptic who defaults to "refuted." It grounds each plan against the tech-spec and the ADRs — hunting contradictions, gaps, and silent assumptions — and every objection is either **FIXED** in a plan or **ACCEPTED** with a named owner. Nothing is silently dropped. Cross-model disagreement is not noise here; it is the entire point.

ATTACK

Refute

a different model, one plan at a time, as a skeptic who didn't write it — default to refuted.

GROUND

Drift?

check each plan against the tech-spec and the ADRs — contradictions, gaps, silent assumptions.

SHARPEN

Fix / Own

every objection **FIXED** in a plan or **ACCEPTED** with a named owner — nothing silently dropped.

GATE

No open objection survives, **and** the fork is decided here as an output of this pass — does the trust boundary wrap the whole system (Fork A · plan-driven) or each unit (Fork B · task-driven)? Cross-model disagreement is the point: a model won't refute itself hard enough.

One big gate, or many small ones?

THE FORK · DECIDED AT THE END OF CONSENSUS

Pass 4 ends by deciding one thing: *where does the trust boundary sit?* Every pass before it lowered altitude in a straight line. Here the line splits. The question is how large a unit of work you are willing to trust at once — both answers are valid Converge, and both reconverge on the identical shared gate.

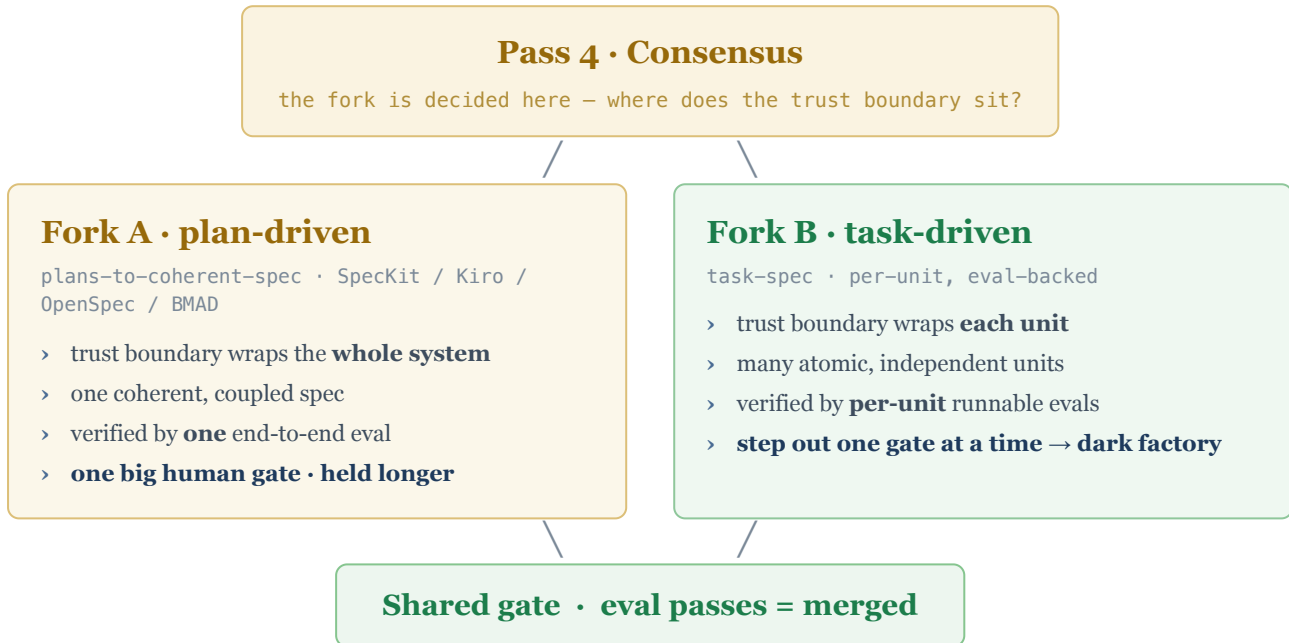


FIG 2 Fork A holds one big human gate; Fork B steps out one small gate at a time — both reconverge on the identical shared gate: one method with two paths, not two.

WHEN TO TAKE FORK A

The pieces only make sense together

Choose plan-driven when the system is **tightly coupled** — no single piece is independently shippable, and "done" is only meaningful for the whole.

- a cross-cutting migration where every table must land together
- one coherent user-facing flow spanning many components
- the e2e eval is the **only** honest test — units passing alone would lie
- you want **one** review moment, held longer, over the whole design
- a named SDD framework already hosts it

WHEN TO TAKE FORK B

The work is loosely coupled & each unit stands alone

Choose task-driven when units are **independently verifiable** — each can be built, proved, and merged on its own without waiting on the others.

- a backlog of features that don't share a trust boundary
- each unit has a cheap, deterministic eval it can own
- you want to remove the human **incrementally**, one gate at a time
- you intend to reach the dark factory — a fleet closing gates on a schedule
- work should parallelise across engines (Claude / Codex / Kimi)

THE TIE-BREAKER

Ask: *if one unit passed its eval and the rest failed, would that be meaningful progress?* If yes → Fork B. If a lone green unit would mislead because the value only exists in the whole → Fork A. Either path reaches the dark factory; only **Fork B** gets there one gate at a time.

5A

Lift the swimlane plans into one coherent spec.

PASS 5A OF 8 · ALTITUDE: COUPLED SPEC · *specify · plan-driven · one coherent spec*

ENGINE · SPECKIT / KIRO / OPENSPEC / BMAD

SKILL · plans-to-coherent-spec

OUT · ONE SPEC · ONE E2E EVAL

Take this fork when the pieces only make sense together — a tightly-coupled system where the trust boundary wraps the whole. The reviewed swimlane plans fold into one coherent, coupled specification — the named SDD frameworks (SpecKit, Kiro, OpenSpec, BMAD) live here, and the engine builds against the spec in one piece, verified by a single end-to-end eval. Because the unit of trust is large, a human holds one big gate and releases it as a unit.

LIFT

Plans → spec

the reviewed swimlane plans fold into one coherent, coupled specification the engine builds in one piece.

COUPLE

One system

the trust boundary wraps the whole — pieces only make sense together, verified as a unit.

BIND EVAL

End-to-end

one eval defines done: sources → gold → the serving layer answers, verified as one pipeline.

GATE

One coherent spec exists, the whole system is its unit of trust, and a single end-to-end eval defines done. Because the unit of trust is large and coupled, a human holds one big gate and releases it as a unit — HITL held longer. This path then enters the shared gate at Execution.

5B

Atomic. Self-contained. Engine-agnostic.

PASS 5B OF 8 · ALTITUDE: ATOMIC UNIT · *tasking* · *task-driven* · *cut into eval-backed units*

ENGINE · TASK-SPEC SKILL

SKILL · task-spec

OUT · *tasks/** — EACH WITH AN EVAL

Take this fork for loosely-coupled work that each stands alone. The plans decompose into many atomic, eval-backed task-specs — the trust boundary wraps each unit, not the system, and verification is per-unit. Because the gates are granular, trust is earned one unit at a time: each eval that goes green is a gate you no longer hold by hand. This is the path that reaches the Dark Factory — many small, automatable gates a fleet can close on a schedule.

CUT

Atomize

one indivisible unit per task — the smallest thing that can be built and verified on its own.

DESCRIBE

Tech, not agent

name the work and paths, not the harness — keep each task readable without the harness.

BIND EVAL

Done = ?

a runnable eval defines done — the test passes, the query returns N, the endpoint responds.

GATE

Every task carries a runnable eval — no eval, not a task yet. The trust boundary wraps each unit; verification is per-unit. Task-spec is self-contained and vendor-portable — Claude, Codex, Kimi, or manual — and the sign-off is a key-optional HMAC seal, so hand-stamping is rejected.

①

Each Task-Spec becomes one tracked issue.

REGISTER · *the tracker as state*

SKILL · task-specs-to-issues

--tracker linear (github | jira)

OUT · 1 SPEC = 1 ISSUE

When tasks leave Tasking, you optionally register each Task-Spec as exactly one tracker issue — its state shadow. The 1:1 mapping is the whole mechanic: one spec, one issue, no drift. The tracker is a **pluggable** backend behind a two-method adapter — read ready issues / write result — so GitHub Issues, Linear, or Jira are interchangeable slots, not commitments. `blocked-by` links carry the dependency graph: 42 `blocked-by` 41 puts the build order into the tracker itself, so state can be read instead of inferred.

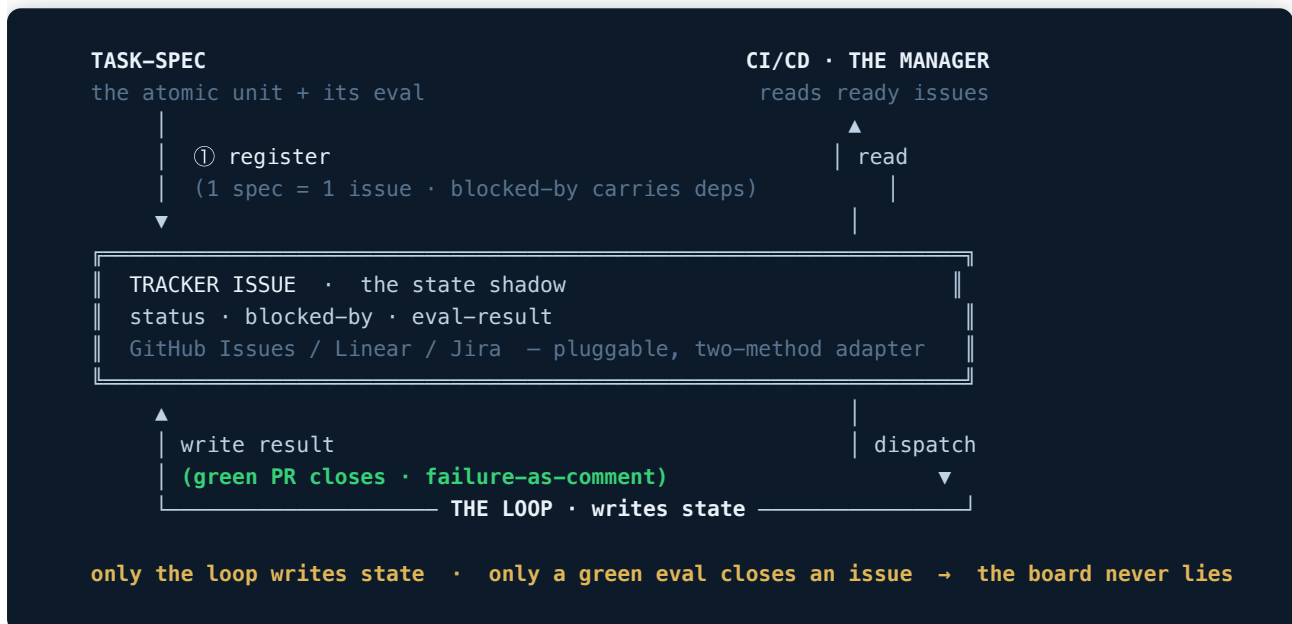


FIG 3 The tracker is the shared state both loops read and write — one Task-Spec registers as one issue; CI/CD reads ready issues, the loop writes results. Only a green eval closes an issue, so the board is state you can dispatch against unattended.

This is what makes the board state **perfect**. Only the loops write state — never a human nudging a card. Only a green eval moves an issue to done — never a feeling, never an attempt-count. Because every transition is mechanical and eval-gated, the board never lies: what it shows is what is true. A "done" column you can trust is the precondition for the Manager loop dispatching against it unattended.

REPO-VS-TRACKER IS A REAL CHOICE, NOT AN UPGRADE

Stay at task-spec (repo files) when you want repo-local control — the specs travel with the code, version with the commit, need no external system. Add task-specs-to-issues when you want a board the loops read across sessions and machines. The tracker is that shared memory; the repo files are the source.

6

Build the rails the work rides on — the quality moat.

PASS 6 OF 8 · ALTITUDE: CONTROL LAYER · *harness* · *generate the build-system the stack needs*

ENGINE · AGENTS-KBS-TECH-STACK

SKILL · stack-to-harness

OUT · `.claude/` + `AGENTS.md`

Pass 6 binds the harness to the **system's tech stack** — not to the tasks in flight. The dbt knowledge base, the MCP server agent, the DuckDB ruleset exist because the system runs dbt, MCP, and DuckDB — not because ISSUE-41 or ISSUE-42 named them. The tasks *scope* which stack components are in play, so you scaffold only what the system actually uses, never speculatively. The stack fills the harness; the tasks point at it. Tasking precedes Harness for exactly this reason: the work reveals which slice of the stack is in scope.

SCOPE

Read stack

the tasks reveal which techs are in play (dbt + duckdb + mcp) — scaffold exactly those, not speculatively.

SCAFFOLD

Agents + KBs

paired architect + developer per tech, grounded in real docs, plus KBs and rules.

EMIT

Cross-tool

AGENTS.md · Cursor · Copilot
— every engine inherits one contract.

GATE

The control plane stands and is grounded in the system's tech stack — the tasks scope which components are in play, the stack fills the content. This is the quality moat: the files that make any commodity agent succeed, and the one artifact that outlives the project.

8

Build the execution loop. Schedule the Manager later.

PASS 8 · THE LOOP — *loop engineering*

Loop Engineering is the discipline of designing the loops that prompt agents, rather than prompting agents. The unit of work is no longer the message but the cycle. One primitive recurs everywhere — take an action, read the feedback the environment returns, decide the next move, repeat until the **goal condition** holds. Convergence is defined by the goal, never by attempt-count: a loop that stops because it tried three times has not converged; a loop that stops because the eval is green has.

Pass 8 · The Loop — task-loop (the execution loop). This is the loop you build. It takes ONE issue (`--issue N`, handed to it by a human or by CI), reads its task-spec plus the cited ADRs and the grounded harness, cuts a branch, and acts — it writes the code. Then it runs the task's own eval. **RED:** it feeds the failure back into itself and revises, a tight local loop that repeats until the eval clears. **GREEN:** it opens a Pull Request that closes the issue.

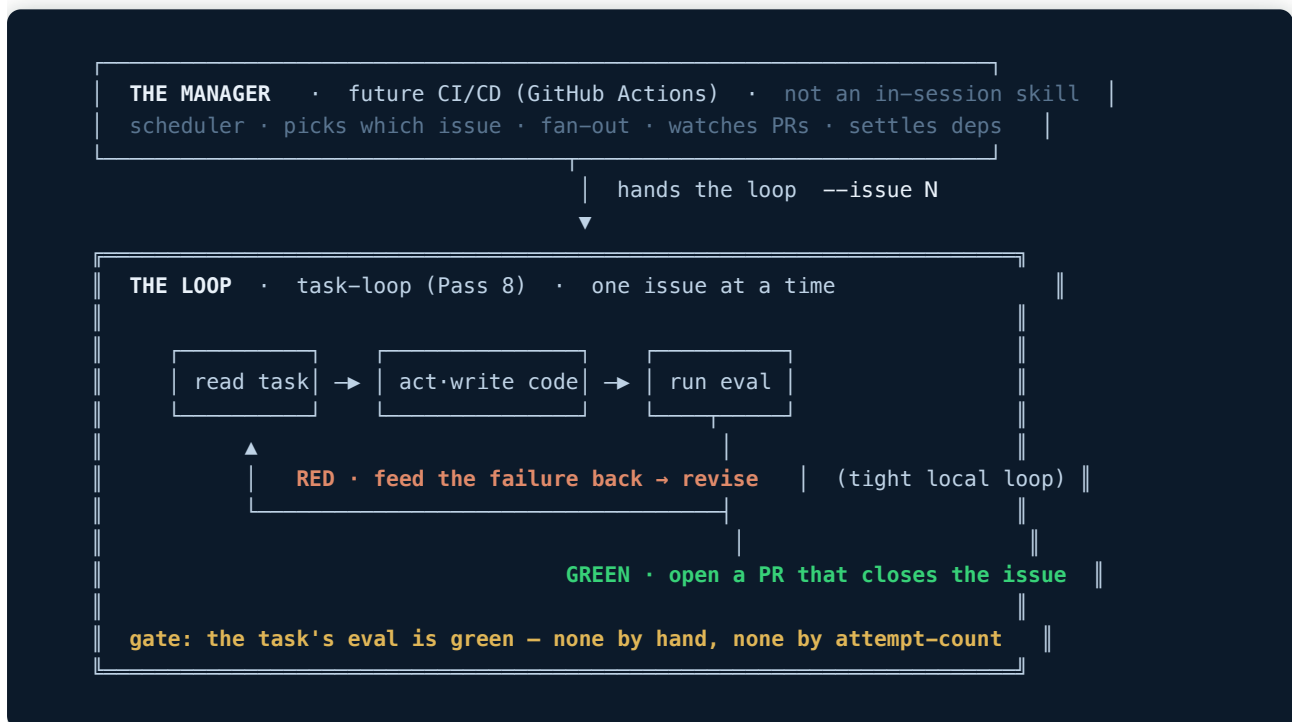


FIG 4 The execution loop — task-loop takes one issue, writes code, runs the task's eval, feeds RED back until GREEN, then opens a PR. The Manager that picks which issue and fans out is a future CI/CD concern (GitHub Actions), not an in-session skill.

GATE

Pass 8 — the task's eval is green; every issue closed by its OWN eval, none by hand, none by attempt-count. Schedule this loop in CI/CD and the human steps out: that is the Dark Factory, one issue at a time.

Why this is defensible

TWO MOATS · POSITIONING · THE HONEST BOUNDARY

The two moats

Converge holds **two** moats, not one, and the discipline is to keep them separate.

OPTIONALITY

Task-Spec

the eval — not the agent — defines done, so you swap Codex / Kimi / Claude freely; the coding agent is a commodity slot.

QUALITY

Harness

the control-plane files that make any commodity agent succeed; not downloadable, built over thousands of hours.

TOGETHER

Defensible

run on commodity models, produce defensible output. Two moats — optionality and quality — not one.

Positioning vs the field

The canonical 2026 SDD pipeline is Specify → Plan → Tasks → Implement, punctuated by human checkpoints — Spec Kit, Kiro, OpenSpec, BMAD all share this shape. Converge is a **loop-closed superset**. It adds the three things those frameworks lack: the adversarial Consensus pass, where a different model attacks each plan; the Fork, which decides where the trust boundary sits; and the closed Loop of Pass 8, which drives a whole fleet to green. The named frameworks stop at Implement with a human standing at the checkpoint. Converge keeps going — and does not replace them: they are exactly the frameworks that host Fork A.

THE HONEST BOUNDARY

The Loop converges to **green evals, not to correct outcomes**. A passing spec test guarantees only that the code matches the spec — if the spec is wrong, the code faithfully implements the wrong thing. The Dark Factory automates the build, not the judgment of whether the spec was right. The defense is structural and partial, not total: Pass 1's gate forces the spec to answer the brief, and Pass 4's adversary attacks it before any code is written. Stating this plainly is the point — the intellectual honesty is the moat's foundation, not a footnote.

The runtime, proven

WHAT THE SKILLS ALREADY ENFORCE — NOT A PROMISE, A MECHANISM

The gates above are not honour-system prompts. The `task-spec` skill ships the runtime that makes "done = proved" a mechanism a human cannot fake and an agent cannot shortcut. Three enforcement layers matter most, and all three already exist on disk.

LAYER	WHAT IT ENFORCES · WHERE IT LIVES
Atomic state machine	A locked, portable transition protocol — advisory lock (mkdir-based, macOS-safe), read current status, validate the transition against a real state machine, write via tmp-file + atomic move. Two agents cannot both claim one <code>ready</code> task. <code>transition-status.sh</code>
Tool-captured evidence	The acceptance gate <i>re-runs the evals itself</i> — it never trusts the executor's word. A non-zero exit is rejected automatically. This is the hole DGE's "evidence gate" fills; Converge fills it too. <code>accept-task.sh</code> · GATE A
Anti-reward-hacking	Beyond re-running: blast-radius check (changed files must be inside <code>touches_paths</code>), HMAC contract integrity (evals were not weakened after sign-off), and gold-sanity (the eval must FAIL on the unpatched baseline). GATE B · C · E
Rebuildable ledger	The T-spec frontmatter is truth; <code>_metrics.jsonl</code> is an append-only ledger; <code>_state.yaml</code> is a rebuildable index. The board is a projection of one source of truth — it cannot silently diverge. <code>rebuild-state.sh</code>

THE ONE GAP WORTH CAPTURING FROM THE FIELD

Converge's eval verdict is binary — green or red. A useful third state exists that it does not yet model: **ambiguous** — the eval ran but cannot prove the claim. Adding an `ambiguous` verdict that never satisfies a gate is the single cheapest, most honest upgrade to the acceptance contract. It is atomic, falsifiable, and Task-Spec-shaped today.

WHAT THIS MEANS FOR AUTONOMY

Autonomy is the **reward** for understanding, not a substitute for it. Converge earns the right to run dark at Pass 8 by spending human judgment at Passes 1–4 — the swimlanes, the ADRs, the adversary. A running loop with no upstream comprehension builds the wrong thing flawlessly. The factory must run dark *and* be aimed correctly; these passes are what aim it.

One method, every engineer

THE ROLE LIVES IN THE EVAL, NOT THE LOOP

The loop never inspects what a task is — it inspects only one thing: does the task carry a runnable eval, and is that eval green. All role-specificity is pushed down into the per-task eval. The loop primitive — take an action, read the environment, decide, repeat until the goal condition holds — is identical whether the artifact is a SQL model, a service, an IaC module, or a retriever. That is precisely why Converge is canonical across disciplines: the role does not change the machine, it only changes the assertion the machine waits on.

ROLE	WHAT ITS TASK'S EVAL IS
Data Engineer	a dbt test + a control-sum query against gold
Software Engineer	a unit / integration test that passes
DevOps	terraform plan clean + a smoke test
AI Engineer	a retrieval-precision or eval-set threshold met

THE PRINCIPLE

The role lives in the eval, not the loop. `task-loop` runs whatever until its eval is green — and CI/CD schedules it — both role-agnostic by construction. That is why one method serves Data Engineering, Software, DevOps, and AI Engineering alike.

The method, end to end on one feature

POSTGRES → DUCKDB → DBT → MCP · TASK-DRIVEN

One feature carries the whole framework: *"daily revenue per product category, served via MCP so a non-engineer can ask for it."* The real metric is `SUM(payments.amount) WHERE payments.status='paid'`, grouped by `products.category` and `date(orders.ordered_at)` in UTC — every pass on the real repo.

PASS	WHAT HAPPENS ON THE REAL REPO
P1 · Intent	BRD → tech-spec: one gold model + one MCP tool over raw orders + payments + products. <i>Gate: the spec answers the brief.</i>
P2 · Structure	Brownfield. Open <code>src/db/01_schema.sql</code> ; write ADRs 0001-join-on-order-id, 0002-utc-date-grain, 0003-revenue-is-paid-only. System understood vs reality.
P3 · Decompose	Two plans, plan-altitude only: Plan A transform (silver → gold_daily_revenue), Plan B serve (mcp tool query_daily_revenue).
P4 · Consensus	<code>--adversary codex</code> attacks: <i>"midnight off-by-one (no TZ)? refunds counted?"</i> Fix: pin UTC (ADR-0002). Accept: refunds out of v1. Fork → task-driven .
P5B · Tasking	<code>tasks/build_gold_revenue</code> (eval: dbt test + control-sum) and <code>tasks/build_mcp_revenue</code> (eval: tool result == gold query). Each born with its eval.
① Register	<code>--tracker linear</code> : build_gold_revenue → ISSUE-41 [ready]; build_mcp_revenue → ISSUE-42 [blocked-by 41]. The board is shared state.
P6 · Harness	Stack = dbt + duckdb + mcp → scaffold <code>.claude/</code> with dbt-architect, mcp-developer, KBs, AGENTS.md. The control plane stands. This is the quality moat.
P8 · The Loop	<code>task-loop --issue 41</code> : dbt test RED (null categories) → add COALESCE → GREEN + control-sum matches → PR; 41 done. Unblocks 42 → eval green → PR; 42 done.

THE DARK FACTORY, ON THIS REPO

A non-engineer asks the MCP *"revenue by category yesterday?"* — and it answers. Built brief → deploy, every step eval-verified, the human walked out at the gate. That last step is the Dark Factory, running on this repo.